🚀 TOPIC 19.0: ADVANCED FOUNDATIONS & TRADE-OFF COMPARISON

🔗 See Topic 18 → Mathematics Masterclass

🎯 **GOAL:** Master Senior & Staff Level Data Structures! Learn Segment Trees with Lazy Propagation, Fenwick Trees (BIT), Sparse Tables, KMP, Rabin-Karp, Z-Algorithm, Manacher's Algorithm, and Tarjan's SCC/Bridges!

🧰 REUSABLE ADVANCED DATA STRUCTURE UTILITIES

Fenwick Tree (BIT) Operations:

```
void update(int[] bit, int i, int val) {
    for (; i < bit.length; i += i & -i) bit[i] += val;
}
int query(int[] bit, int i) {
    int sum = 0;
    for (; i > 0; i -= i & -i) sum += bit[i];
    return sum;
}
```

Segment Tree Array Size Rule:

```
// Always allocate 4 * N for Segment Tree array
int[] tree = new int[4 * n];
```

💡 **AHA MOMENT #1: ARCHITECTURAL RANGE QUERY TRADE-OFF MATRIX**

Choosing the right structure for Range Queries ($[L, R]$) and Updates:

- **Prefix Array:** $O(1)$ Query, $O(N)$ Update (Static array only).
- **Fenwick Tree (BIT):** $O(\log N)$ Query, $O(\log N)$ Point Update. Simple code, $O(N)$ memory!
- **Segment Tree:** $O(\log N)$ Query, $O(\log N)$ Range Update with Lazy Prop! Supports Min/Max/GCD/Sum.
- **Sparse Table:** $O(1)$ RMQ Query, $O(N \log N)$ Static Build (No updates allowed!).

📊 RANGE QUERY DATA STRUCTURE COMPARISON TABLE

Data Structure	Build Time	Query Time	Update Time	Space	Supported Functions
Prefix Sum Array	$O(N)$	$O(1)$	$O(N)$	$O(N)$	Sum only (Static)
Fenwick BIT	$O(N)$	$O(\log N)$	$O(\log N)$	$O(N)$	Sum, Prefix Min/Max
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$	$O(4N)$	Sum, Min, Max, GCD (Associative)
Sparse Table	$O(N \log N)$	$O(1)$	∞ (Static)	$O(N \log N)$	Min, Max, GCD (Idempotent)

📌 **KEY TAKEAWAY:** Use Fenwick BIT when only point updates & prefix sums are needed; use Segment Tree when range updates or complex functions (min/max/gcd) are required!

🌲 TOPIC 19.1: SEGMENT TREE & LAZY PROPAGATION

💡 **AHA MOMENT #2: LAZY PROPAGATION DEFERRED UPDATES**

When updating a range $[qL, qR]$, updating all leaf nodes takes $O(N)$ time.

Lazy Propagation: Store pending updates in `lazy[node]`. Update current node's value immediately, and push lazy update to children ONLY when visiting them later! This keeps range updates at $O(\log N)$ time!

🛠️ SEGMENT TREE WITH LAZY PROPAGATION IMPLEMENTATION

```
class SegmentTree {
    int[] tree, lazy; int n;
    public SegmentTree(int[] nums) {
        n = nums.length; tree = new int[4 * n]; lazy = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }
    private void build(int[] nums, int node, int start, int end) {
        if (start == end) { tree[node] = nums[start]; return; }
        int mid = (start + end) / 2;
        build(nums, 2 * node + 1, start, mid);
        build(nums, 2 * node + 2, mid + 1, end);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }
    public void updateRange(int node, int start, int end, int l, int r, int val) {
        if (lazy[node] ≠ 0) { // Push pending lazy update!
            tree[node] += (end - start + 1) * lazy[node];
            if (start ≠ end) { lazy[2*node+1] += lazy[node]; lazy[2*node+2] += lazy[node]; }
            lazy[node] = 0;
        }
        if (start > end || start > r || end < l) return;
        if (start ≥ l && end ≤ r) {
            tree[node] += (end - start + 1) * val;
            if (start ≠ end) { lazy[2*node+1] += val; lazy[2*node+2] += val; }
            return;
        }
        int mid = (start + end) / 2;
        updateRange(2*node+1, start, mid, l, r, val);
        updateRange(2*node+2, mid+1, end, l, r, val);
        tree[node] = tree[2*node+1] + tree[2*node+2];
    }
}
```

📌 **KEY TAKEAWAY:** Always size `tree` and `lazy` arrays to `4 * N` to prevent `ArrayIndexOutOfBoundsException`!

TOPIC 19.2: FENWICK TREE (BINARY INDEXED TREE / BIT)

💡 **AHA MOMENT #3: LOWBIT MATH FOR $O(\log N)$ PREFIX SUMS**

Every integer index can be decomposed into powers of 2.

- **Lowest Set Bit Math:** `lowbit(i) = i & -i`
- **Update `i`:** `i += i & -i` (Traverse up parents to propagate change).
- **Query Prefix `i`:** `i -= i & -i` (Traverse back to aggregate prefix sums).

Uses 1-based indexing array of size `N + 1`!

🛠️ FENWICK TREE (BIT) $O(\LOG N)$ IMPLEMENTATION

```
class FenwickTree {
    private int[] tree;
    public FenwickTree(int n) { tree = new int[n + 1]; }
    public void add(int i, int delta) {
        for (; i < tree.length; i += i & -i) tree[i] += delta;
    }
    public int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
    public int queryRange(int left, int right) {
        return query(right) - query(left - 1);
    }
}
```

📌 **KEY TAKEAWAY:** Fenwick BIT requires only 10 lines of code and $O(N)$ space!

📊 TOPIC 19.3: SPARSE TABLE (IMMUTABLE RMQ IN O(1))

💡 **AHA MOMENT #4: OVERLAPPING $O(1)$ RANGE MINIMUM QUERIES**

For static arrays (no updates), Sparse Table precomputes `st[i][j]` = RMQ for range $[i, i + 2^j - 1]$.
Query $[L, R]$ in $O(1)$ time by overlapping two power-of-2 intervals:
`k = log2(R - L + 1)`
`ans = Math.min(st[L][k], st[R - (1 << k) + 1][k])`

🛠️ SPARSE TABLE $O(N \LOG N)$ BUILD, $O(1)$ QUERY

```
class SparseTable {
    int[][] st; int[] log;
    public SparseTable(int[] nums) {
        int n = nums.length; int maxLog = 32 - Integer.numberOfLeadingZeros(n);
        st = new int[n][maxLog]; log = new int[n + 1];
        for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;
        for (int i = 0; i < n; i++) st[i][0] = nums[i];
        for (int j = 1; j < maxLog; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[i][j] = Math.min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
            }
        }
    }
    public int queryMin(int L, int R) {
        int k = log[R - L + 1];
        return Math.min(st[L][k], st[R - (1 << k) + 1][k]);
    }
}
```

📌 **KEY TAKEAWAY:** Sparse Table achieves $O(1)$ RMQ because `min` is an idempotent operation ($\min(A, A) = A$)!

🗺️ TOPIC 19.4: JAVA ORDERED DATA STRUCTURES (`TREEMAP` & `TREESET`)

💡 **AHA MOMENT #5: RED-BLACK TREE NAVIGABLE APIS**

Java `TreeMap` and `TreeSet` implement self-balancing Red-Black BSTs:

- `floorKey(k)` ➔ Greatest key $\le k$.
- `ceilingKey(k)` ➔ Smallest key $\ge k$.
- `lowerKey(k)` ➔ Greatest key $< k$.
- `higherKey(k)` ➔ Smallest key $> k$.

All operations execute in guaranteed $O(\log N)$ time!

🛠️ MY CALENDAR I (LC 729) TREEMAP OVERLAP CHECK

```
class MyCalendar {
    TreeMap<Integer, Integer> calendar = new TreeMap<>();
    public boolean book(int start, int end) {
        Integer prev = calendar.floorKey(start);
        Integer next = calendar.ceilingKey(start);
        if ((prev ≠ null && calendar.get(prev) > start) ||
            (next ≠ null && next < end)) {
            return false; // Overlap detected!
        }
        calendar.put(start, end);
        return true;
    }
}
```

📌 **KEY TAKEAWAY:** `TreeMap` enables $O(\log N)$ interval overlap queries without manual BST coding!

💡 **AHA MOMENT #6: KMP PREFIX FUNCTION (`LPS[]`)**
KMP (Knuth-Morris-Pratt) precomputes `lps[i]` = length of longest proper prefix of pattern that is also a suffix of `pattern[0...i]`.
When a mismatch occurs at `pattern[j]`, shift pattern index `j = lps[j - 1]` without rewinding the text pointer `i`!
Achieves linear $O(N + M)$ pattern matching!

🛠️ **KMP PATTERN MATCHING (LC 28) $O(N + M)$ IMPLEMENTATION**

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int[] lps = buildLPS(needle);
    int i = 0, j = 0;
    while (i < haystack.length()) {
        if (haystack.charAt(i) == needle.charAt(j)) { i++; j++; }
        if (j == needle.length()) return i - j; // Match found!
        else if (i < haystack.length() && haystack.charAt(i) ≠ needle.charAt(j)) {
            if (j ≠ 0) j = lps[j - 1]; // Shift via LPS!
            else i++;
        }
    }
    return -1;
}
private int[] buildLPS(String pattern) {
    int[] lps = new int[pattern.length()];
    int len = 0, i = 1;
    while (i < pattern.length()) {
        if (pattern.charAt(i) == pattern.charAt(len)) lps[i++] = ++len;
        else if (len ≠ 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
```

📌 **KEY TAKEAWAY:** KMP never backtracks text pointer `i`, guaranteeing linear $O(N + M)$ search!

🎀 TOPIC 19.5: ADVANCED STRINGS (Z-ALGO & MANACHER) PAGE 7 OF 16

💡 **AHA MOMENT #7: MANACHER'S $O(N)$ LONGEST PALINDROME SEARCH**
1. Transform string `s` by inserting `#` between characters (`"aba" → "#a#b#a#"`).
2. Maintain palindrome center `C` and right boundary `R`.
3. Re-use previously computed palindrome radii using mirror index `mirror = 2*C - i`!
Finds longest palindromic substring in exact $O(N)$ linear time!

🛠️ MANACHER'S ALGORITHM (LC 5) $O(N)$ IMPLEMENTATION

```
public String longestPalindrome(String s) {
    StringBuilder sb = new StringBuilder("#");
    for (char c : s.toCharArray()) sb.append(c).append("#");
    String t = sb.toString();
    int n = t.length(), C = 0, R = 0, maxLen = 0, centerIndex = 0;
    int[] P = new int[n];
    for (int i = 0; i < n; i++) {
        int mirror = 2 * C - i;
        if (i < R) P[i] = Math.min(R - i, P[mirror]);
        while (i + 1 + P[i] < n && i - 1 - P[i] >= 0 &&
               t.charAt(i + 1 + P[i]) == t.charAt(i - 1 - P[i])) P[i]++;
        if (i + P[i] > R) { C = i; R = i + P[i]; }
        if (P[i] > maxLen) { maxLen = P[i]; centerIndex = i; }
    }
    int start = (centerIndex - maxLen) / 2;
    return s.substring(start, start + maxLen);
}
```

📌 **KEY TAKEAWAY:** Manacher's `#` transformation unifies odd and even length palindrome expansion!

📜 TOPIC 19.8 & 1-PAGE PRINTABLE ADVANCED DS CHEAT SHEET

🌳 ADVANCED DATA STRUCTURE DECISION TREE (MERMAID)

📜 PRINTABLE ADVANCED DATA STRUCTURE CHEAT SHEET

Structure / Algorithm	Key Technique / Formula	Time Complexity	Space
Fenwick BIT	`idx += idx & -idx` / `idx -= idx & -idx`	$O(\log N)$ Query & Update	$O(N)$
Segment Tree + Lazy Prop	`lazy[node]` deferred range updates	$O(\log N)$ Range Query & Update	$O(4N)$
Sparse Table	`min(st[L][k], st[R - (1<	$O(1)$ Query, $O(N \log N)$ Build	$O(N \log N)$
Java TreeMap	Red-Black BST `floorKey()`, `ceilingKey()`	$O(\log N)$ Search & Insert	$O(N)$
KMP Search	`lps[]` Longest Prefix Suffix array	$O(N + M)$ Search	$O(M)$
Manacher Palindrome	`#` padding + mirror index `2*C - i`	$O(N)$ Linear Search	$O(N)$

🗣️ **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

💻 ADVANCED TEMPLATE LIBRARY (SEGMENT TREE & BIT)

1. FENWICK TREE (BIT) TEMPLATE

```
class BIT {
    int[] tree;
    BIT(int n) { tree = new int[n + 1]; }
    void add(int i, int delta) { for (; i < tree.length; i += i & -i)
tree[i] += delta; }
    int query(int i) { int sum = 0; for (; i > 0; i -= i & -i) sum +=
tree[i]; return sum; }
    int queryRange(int l, int r) { return query(r) - query(l - 1); }
}
```

2. SPARSE TABLE (RMQ) TEMPLATE

```
class SparseTable {
    int[][] st; int[] log;
    SparseTable(int[] a) {
        int n = a.length, m = 32 - Integer.numberOfLeadingZeros(n);
        st = new int[n][m]; log = new int[n + 1];
        for (int i = 2; i <= n; i++) log[i] = log[i/2] + 1;
        for (int i = 0; i < n; i++) st[i][0] = a[i];
        for (int j = 1; j < m; j++)
            for (int i = 0; i + (1<<j) <= n; i++)
                st[i][j] = Math.min(st[i][j-1], st[i+(1<<(j-1))][j-
1]);
    }
    int query(int l, int r) { int k = log[r - l + 1]; return
Math.min(st[l][k], st[r-(1<<k)+1][k]); }
}
```

📌 **REUSABLE CODE:** Copy-paste these templates directly into your interview whiteboard or IDE!

3. KMP PATTERN MATCH TEMPLATE

```
int[] buildLPS(String p) {
    int[] lps = new int[p.length()]; int len = 0, i = 1;
    while (i < p.length()) {
        if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
        else if (len != 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
```

4. MANACHER PALINDROME TEMPLATE

```
int[] manacher(String s) {
    StringBuilder sb = new StringBuilder("#");
    for (char c : s.toCharArray()) sb.append(c).append("#");
    String t = sb.toString(); int n = t.length(), C = 0, R = 0;
    int[] P = new int[n];
    for (int i = 0; i < n; i++) {
        if (i < R) P[i] = Math.min(R - i, P[2 * C - i]);
        while (i + 1 + P[i] < n && i - 1 - P[i] >= 0 && t.charAt(i + 1
+ P[i]) == t.charAt(i - 1 - P[i])) P[i]++;
        if (i + P[i] > R) { C = i; R = i + P[i]; }
    }
    return P;
}
```

📌 **KEY TAKEAWAY:** Memorize these core templates to solve senior-level FAANG interview questions!

REPRESENTATIVE SUITE: RANGE SUM QUERY - MUTABLE

1. LEETCODE 307 — RANGE SUM QUERY - MUTABLE

Medium ★★★★★ Fenwick BIT / Segment Tree Amazon Google

Problem:

Calculate range sum query and support point updates in $O(\log N)$ time.

```
class NumArray {
    private int[] tree, nums;
    public NumArray(int[] nums) {
        this.nums = nums;
        tree = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) add(i + 1, nums[i]);
    }
    private void add(int i, int val) {
        for (; i < tree.length; i += i & -i) tree[i] += val;
    }
    public void update(int index, int val) {
        int diff = val - nums[index];
        nums[index] = val;
        add(index + 1, diff);
    }
    private int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
    public int sumRange(int left, int right) {
        return query(right + 1) - query(left);
    }
}
```

AHA MOMENT (LC 307): Fenwick BIT updates and queries range sum in $O(\log N)$ time with only 20 lines of code!

2. LEETCODE 28 — FIND FIRST OCCURRENCE IN STRING (KMP)

Easy ★★★★★ KMP Prefix Function Amazon Meta

Problem:

Return index of first occurrence of `needle` in `haystack` in $O(N + M)$ time.

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int[] lps = new int[needle.length()];
    int len = 0, i = 1;
    while (i < needle.length()) {
        if (needle.charAt(i) == needle.charAt(len)) lps[i++] = ++len;
        else if (len != 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    i = 0; int j = 0;
    while (i < haystack.length()) {
        if (haystack.charAt(i) == needle.charAt(j)) { i++; j++; }
        if (j == needle.length()) return i - j;
        else if (i < haystack.length() && haystack.charAt(i) !=
needle.charAt(j)) {
            if (j != 0) j = lps[j - 1];
            else i++;
        }
    }
    return -1;
}
```

💡 **AHA MOMENT (LC 28):** `lps[j - 1]` allows pattern pointer `j` to jump directly to the longest matching prefix without re-checking matched characters!

3. LEETCODE 5 — LONGEST PALINDROMIC SUBSTRING (MANACHER)

🟡 Medium ★★★★★ Manacher $O(N)$ Linear Search Amazon Google

Problem:

Return the longest palindromic substring in $O(N)$ linear time.

```
public String longestPalindrome(String s) {
    StringBuilder sb = new StringBuilder("#");
    for (char c : s.toCharArray()) sb.append(c).append("#");
    String t = sb.toString();
    int n = t.length(), C = 0, R = 0, maxLen = 0, centerIndex = 0;
    int[] P = new int[n];
    for (int i = 0; i < n; i++) {
        int mirror = 2 * C - i;
        if (i < R) P[i] = Math.min(R - i, P[mirror]);
        while (i + 1 + P[i] < n && i - 1 - P[i] >= 0 &&
               t.charAt(i + 1 + P[i]) == t.charAt(i - 1 - P[i])) P[i]++;
        if (i + P[i] > R) { C = i; R = i + P[i]; }
        if (P[i] > maxLen) { maxLen = P[i]; centerIndex = i; }
    }
    int start = (centerIndex - maxLen) / 2;
    return s.substring(start, start + maxLen);
}
```

💡 **AHA MOMENT (LC 5):** Manacher's algorithm outperforms $O(N^2)$ expand-around-center by leveraging palindrome symmetry!

4. LEETCODE 218 — THE SKYLINE PROBLEM

Hard ★★★★★ Sweep Line + TreeMap Max Height Amazon Meta

Problem:

Return key points that outline the skyline formed by all buildings.

```
public List<List<Integer>> getSkyline(int[][] buildings) {
    List<int[]> events = new ArrayList<>();
    for (int[] b : buildings) {
        events.add(new int[]{b[0], -b[2]}); // Start event (-height)
        events.add(new int[]{b[1], b[2]});  // End event (+height)
    }
    events.sort((a, b) -> a[0] != b[0] ? Integer.compare(a[0], b[0]) :
Integer.compare(a[1], b[1]));
    List<List<Integer>> res = new ArrayList<>();
    TreeMap<Integer, Integer> heights = new TreeMap<>();
    heights.put(0, 1); int prevMax = 0;
    for (int[] e : events) {
        int x = e[0], h = e[1];
        if (h < 0) heights.put(-h, heights.getOrDefault(-h, 0) + 1);
        else {
            if (heights.get(h) == 1) heights.remove(h);
            else heights.put(h, heights.get(h) - 1);
        }
        int currMax = heights.lastKey();
        if (currMax != prevMax) {
            res.add(Arrays.asList(x, currMax)); prevMax = currMax;
        }
    }
    return res;
}
```

FAANG COACH TIP: `heights.lastKey()` gets the maximum active building height in $O(\log N)$ time!

5. LEETCODE 1192 — CRITICAL CONNECTIONS IN A NETWORK (TARJAN)

Hard ★★★★★ Tarjan Bridge Detection Amazon Meta

Problem:

Return all critical connections (bridges) whose removal disconnects the graph.

```
class Solution {
    private int timer = 1;
    public List<List<Integer>> criticalConnections(int n,
List<List<Integer>> connections) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (List<Integer> edge : connections) {
            adj.get(edge.get(0)).add(edge.get(1));
            adj.get(edge.get(1)).add(edge.get(0));
        }
        int[] disc = new int[n], low = new int[n];
        List<List<Integer>> bridges = new ArrayList<>();
        dfs(0, -1, adj, disc, low, bridges);
        return bridges;
    }
    private void dfs(int u, int p, List<List<Integer>> adj, int[] disc,
int[] low, List<List<Integer>> res) {
        disc[u] = low[u] = timer++;
        for (int v : adj.get(u)) {
            if (v == p) continue;
            if (disc[v] == 0) {
                dfs(v, u, adj, disc, low, res);
                low[u] = Math.min(low[u], low[v]);
                if (low[v] > disc[u]) res.add(Arrays.asList(u, v)); //
Bridge found!
            } else {
                low[u] = Math.min(low[u], disc[v]);
            }
        }
    }
}
```

📌 **KEY TAKEAWAY:** `low[v] > disc[u]` proves neighbor `v` cannot reach any ancestor of `u` without using edge `(u, v)`!

💎 SENIOR & STAFF ENGINEER NOTES & MASTER REVISION

⭐ THE 4 GOLDEN LAWS OF ADVANCED DATA STRUCTURES

1. **Segment Tree Sizing Law:** Always allocate `4 * N` nodes to prevent out-of-bounds errors.
2. **BIT Indexing Law:** Fenwick BIT uses 1-based indexing (`tree[n + 1]`). Lowbit operations crash on index 0!
3. **KMP Backtrack Law:** Never rewind text pointer `i`. Shift pattern pointer `j = lps[j - 1]`.
4. **Sparse Table Idempotent Law:** Sparse table $O(1)$ RMQ works ONLY for idempotent functions ($\min, \max, \gcd$).

⛔ COMMON ADVANCED DS BUGS & PITFALLS

- **Forgetting Lazy Pushdown:** Visiting child nodes in Segment Tree without pushing pending `lazy[node]` updates corrupts subtrees!
- **Manacher String Index Mapping:** Transformed string index `centerIndex` maps back to original string at `(centerIndex - maxLen) / 2`.

📊 SENIOR & STAFF ENGINEER ADVANCED CONFIDENCE TRACKER

🟡 Med LC 307 Range Sum Query Mutable	🟢 Easy LC 28 KMP Pattern Search	🟡 Med LC 5 Manacher Palindrome
🟡 Med LC 729 My Calendar I	🔴 Hard LC 218 The Skyline Problem	🔴 Hard LC 1192 Tarjan Critical Bridges

👉 **Next Master Topic: Topic 20 → System Design & FAANG Final Interview Toolkit**

🎉 **TOPIC 19 ADVANCED DATA STRUCTURES MASTERCLASS COMPLETE!** Turn the page to **Topic 20: System Design & Interview Toolkit**!